

Informe I (Arquitectura del Computador)

Jesus Contreras (C.I: 31.021.750)

2026

1 Implementaciones en MIPS32

1.1 ¿Cómo se implementa la recursividad en MIPS32? ? ¿Qué papel cumple la pila (sp)?

En MIPS32, la recursividad se implementa utilizando la pila (apuntada por el registro `sp`) para guardar el contexto de cada llamada. En el código proporcionado, podemos observar:

- Al inicio de `paridad_recursiva`, se reservan 8 bytes en la pila: `subi $sp,$sp, 8`
- Se guardan el registro de retorno `$ra` y el argumento `$a0` en la pila: `sw $ra, 4($sp)` y `sw $a0, 0($sp)`
- Esto permite que después de la llamada recursiva, se puedan restaurar estos valores: `lw $a0, 0($sp)` y `lw $ra, 4($sp)`
- Finalmente se libera el espacio de pila: `addi $sp, $sp, 8`

La pila es fundamental porque cada llamada recursiva tiene su propio conjunto de variables locales y dirección de retorno. Sin ella, no sería posible mantener la independencia de cada nivel recursivo.

1.2 ¿Qué riesgos de desbordamiento existen? ¿Cómo mitigarlos?

El principal riesgo es el `stack overflow` (desbordamiento de pila), que ocurre cuando:

- La profundidad de la recursión es muy grande (ej. $n = 100000$)
- Cada llamada consume 8 bytes en la pila, por lo que para $n = 10000$ se usarían 80KB aproximadamente
- En MARS, la pila tiene un tamaño limitado (típicamente 1 MB)

Para mitigarlo:

- Usar enfoques iterativos cuando se prevean números grandes
- Verificar que los casos base estén bien definidos para evitar recursión infinita
- En sistemas críticos, calcular el máximo de llamadas posibles y dimensionar la pila adecuadamente
- Implementar técnicas como recursión de cola (tail recursion) cuando sea posible

1.3 ¿Qué diferencias encuentras entre una implementación iterativa y una recursiva en cuanto al uso de memoria y registros?

- **Uso de memoria:**
 - Recursiva: Utiliza la pila dinámicamente. Cada llamada consume 8 bytes adicionales.
 - Iterativa: No usa memoria dinámica. Solo utiliza registros (\$t0, \$t1, \$t2) y una variable en \$s0.
- **Registros:**
 - Recursiva: Requiere guardar y restaurar \$ra y \$a0 constantemente. Hay mayor movimiento de datos entre registros y memoria.
 - Iterativa: Todo el procesamiento se hace con registros temporales. No hay acceso a memoria más allá de la inicial.

1.4 ¿Qué diferencias encuentras entre los ejemplos académicos del libro y un ejercicio completo y operativo en MIPS32?

Los ejemplos académicos suelen presentar:

- Fragmentos de código aislados y perfectamente comentados
- Suposiciones de que los datos ya están en registros

- Omisión de la interacción con el usuario

Un ejercicio completo como este implica:

- Manejo de entrada/salida con syscalls (imprimir mensajes, leer enteros)
- Gestión correcta de la pila en funciones no triviales
- Preservación de registros según las convenciones de MIPS
- Pruebas con distintos valores y casos límite
- Depuración real en simuladores (MARS) donde se detectan errores de desbordamiento, uso incorrecto de registros, etc.

1.5 Elaborar un tutorial de la ejecución paso a paso en MARS

1. **Abrir MARS** y cargar el archivo `paridad recursiva.asm` (o el iterativo).
2. **Ensamblar** el código: Run → Assemble (F3).
3. En la pestaña Execute, observar el código ensamblado y los segmentos de datos.
4. **Establecer breakpoints:**
 - En la línea `jal paridad_recursiva` (para ver la primera llamada)
 - En la línea `subi $a0, $a0,1` (para ver cada llamada recursiva)
5. **Ejecutar:** Run → Go (F5) e ingresar un número pequeño (ej. 3).
6. Usar Single Step (F7) para avanzar instrucción por instrucción.
7. **Observar:**
 - Cómo cambia `$sp` (stack pointer) en cada llamada
 - Los valores guardados en la pila (ventana Data Segment)
 - El registro `$ra` antes y después de `jal`
 - El resultado final en `$v0`

8. Para la versión iterativa, observar el bucle for y cómo cambian \$t0 y \$t1.

1.6 Justificar la elección del enfoque (iterativo o recursivo) según eficiencia y claridad en MIPS

- **Enfoque recursivo:**

- Claridad: Muy claro conceptualmente, refleja directamente la definición matemática $\text{paridad}(n) = 1 - \text{paridad}(n-1)$.
- Eficiencia: Baja. Cada llamada implica overhead de pila (8 bytes) y acceso a memoria. Para $n=1000$, se realizan 1000 llamadas y se usan 8KB de pila.
- Riesgo: Desbordamiento para n grandes.

- **Enfoque iterativo:**

- Claridad: También claro, aunque la lógica de alternancia ($\text{result} = 1 - \text{result}$) puede no ser intuitiva a primera vista.
- Eficiencia: Alta. Usa solo registros, sin acceso a memoria. Realiza exactamente n iteraciones con operaciones simples.
- Robustez: No hay riesgo de desbordamiento de pila.

Conclusión: Para este problema específico, el enfoque **iterativo es superior** en eficiencia y seguridad, manteniendo una claridad aceptable. El recursivo se justifica principalmente con fines didácticos para aprender manejo de pila y recursividad en ensamblador.

1.7 Análisis y Discusión de los Resultados

- Ambos algoritmos producen el mismo resultado para cualquier entrada positiva. El recursivo implementa correctamente la fórmula, mientras que el iterativo simula el mismo comportamiento mediante un bucle.
- **Rendimiento:** Al probar con números grandes (ej. $n=10000$), el recursivo puede generar desbordamiento de pila en MARS, mientras que el iterativo lo maneja sin problemas.
- **Depuración:** La versión recursiva es más difícil de depurar

debido al gran número de llamadas y la gestión de pila. La versión iterativa permite seguir fácilmente el flujo del programa.

- **Uso de registros:** El recursivo usa \$a0, \$ra y \$sp intensivamente; el iterativo usa \$t0, \$t1, \$t2 y \$s0. El iterativo respeta mejor la convención de uso de registros en MIPS.